

# Critères d'évaluation du projet

## Introduction

Les critères d'évaluation que vous verrez dans ce document sont **seulement** les grands axes de cotation. Son but n'est pas de vous fournir une fiche précise des différents points avec leur pondération, mais plutôt de vous fournir un document qui vous permettra d'évaluer si votre projet est sur la bonne voie ou pas.

Vous trouverez, à la fin de ce document, une liste de bonus à réaliser (ceci est **optionnel**) pour augmenter vos points. Cependant, veuillez noter que chaque bonus suivra le principe du « tout ou rien ». C'est-à-dire que si vous réalisez un bonus à 99%, aucun point ne vous sera donné. Obtenir des points via les critères d'évaluation doit être votre priorité !

Aucune défense orale autorisée sans projet remis. Sans défense orale, la cotation équivaldra à un PR.

## Listes des critères d'évaluation

- Réalisation des fonctionnalités annoncées dans le dossier
- Respect des caractéristiques d'une API : routes bien nommées, séparation des ressources judicieuses, etc.
- Code propre (structure du projet + code des fichiers) : Est-ce que votre projet est structuré ? Est-ce que votre code est "propre" (exemple de code pas propre : un fichier avec 2000 lignes de code) ? Utilisez-vous les dernières fonctionnalités de l'ECMAScript judicieusement ? Etc.
- Gestion correcte des async/await
- Utilisation correcte du JWT : Il est **obligatoire** d'utiliser le JWT dans le projet. Vous serez évalués sur le fait que vous gérez l'identification et l'autorisation avec un JWT. De plus, le JWT ne devra pas contenir ni trop de données, ni trop peu.
- Utilisation correcte des transactions : si vous devez faire plusieurs opérations avec la base de données, utilisez-vous des transactions ? Gérez-vous les cas avec rollbacks ? Etc.
- Un CRUD complet par table et pour 4 tables minimum. Vous pouvez en faire plus, bien sûr.
- API sécurisée : Gestion des injections SQL, utilisation d'un fichier «.env» pour les données sensibles, etc.
- Présence d'un logging/monitoring
- Bonne utilisation des codes HTTP
- Versionning
- Documentation (Swagger) : il vous sera demandé de documenter **entièrement 2 "ressources"**. Cela signifie que vous devrez avoir au moins une requête GET, PATCH/PUT, POST et DELETE concernant la ressource pour qu'elle compte. De plus, il est nécessaire qu'au moins une requête nécessite un JWT pour que la ressource compte.
- Délai de réponses correct
- Tests Postman : Vous devrez avoir 10 requêtes correctes au minimum dans la collection. Chaque requête devra être correctement testée (avoir la bonne série de tests). Dans les 10 requêtes, vous devrez tester un CRUD sur une route protégée (lecture des données, mise à jour des données, insertion de données et suppression de données sur une route protégée).

Dans les 10 requêtes, ne seront pas comptabilisés les requêtes d'identification. Si vous craignez de ne pas avoir 10 requêtes correctes, vous pouvez en faire plus.

### Bonus

- Mise en ligne
- Utilisation du HTTPS
- Rafraichissement des tokens JWT
- « Dockeriser » votre API via un Dockerfile
- « Dockersier » votre système (API + base de données) via « docker-compose » : il faudra un premier container pour votre API et un deuxième container pour la base de données
- Mécanismes utilisés pour accélérer les réponses (exemple : contenu des réponses différent en fonction du header « Accept »). **Attention, ceci est soumis à validation du professeur !**
- Fonctionnalités supplémentaires. **Attention, ceci est soumis à validation du professeur !**
- Utilisation de TypeScript